



PROJECT DOCUMENTATION

Netflix Content Strategy & Genre Analysis

An Intermediate PostgreSQL Project

Analysing Netflix's full content catalogue — genre trends, regional dominance, rating distributions, content addition timelines, and originals vs licensed content.

Prepared By Mounika
Database PostgreSQL 15+
Dataset Size 8,800+ titles
Date 9 June 2026

1. PROJECT OVERVIEW

This project applies intermediate-to-advanced PostgreSQL techniques to perform a structured analysis of Netflix's complete content catalogue. The goal is to answer key strategic questions around how Netflix's content library has evolved over time, which genres dominate specific regional markets, and how the balance between movies and television shows has shifted. The project is designed to develop practical SQL skills through real-world entertainment data.

Problem Statement

"How has Netflix's content strategy shifted over time, and which genres dominate specific regional markets?"

Key Business Questions

- How many titles does Netflix add per year, and what is the year-over-year growth rate?
- What is the content mix between Movies and TV Shows, and has it changed?
- Which genres are most popular globally and within specific countries?
- What is the age/rating distribution of Netflix content?
- Which directors and countries contribute the most content?
- How old is the average licensed content at the time of Netflix addition?
- Which genres tend to appear together in multi-genre titles?

2. DATASET INFORMATION

Source	Kaggle — Netflix Movies and TV Shows
Author	Shivamb (Shivam Bansal)
URL	kaggle.com/datasets/shivamb/netflix-shows
File	netflix_titles.csv (~3 MB, 8,800+ rows)
Licence	CC0: Public Domain
Last Updated	Regularly updated (2021–2023)

Column Reference

Column	Data Type	Description
--------	-----------	-------------

show_id	TEXT	Unique identifier for each title (e.g. s1, s2 ...)
type	TEXT	Content category — 'Movie' or 'TV Show'
title	TEXT	Name of the movie or TV show
director	TEXT	Director name (nullable — many TV shows omit this)
cast_members	TEXT	Comma-separated list of main cast members
country	TEXT	Country or countries where the title was produced
date_added	TEXT→DATE	Date the title was added to Netflix (parsed with TO_DATE)
release_year	INTEGER	Original release year of the content
rating	TEXT	Content rating — TV-MA, PG-13, R, etc.
duration	TEXT	Movie duration in minutes or number of TV seasons
listed_in	TEXT	Comma-separated genres — unnested via STRING_TO_ARRAY
description	TEXT	Short plot summary of the title

3. ENVIRONMENT & SETUP

Prerequisites

- PostgreSQL 13 or above (15+ recommended)
- pgAdmin 4, DBeaver, or any PostgreSQL-compatible client
- Kaggle account (free) to download netflix_titles.csv
- Basic understanding of SQL — SELECT, GROUP BY, JOIN

Database Setup

Step 1 — Create the raw ingestion table

```
CREATE TABLE netflix_raw (  
  show_id      TEXT,  
  type         TEXT,  
  title        TEXT,  
  director     TEXT,  
  cast_members TEXT,  
  country      TEXT,  
  date_added   TEXT,  
  release_year INT,  
  rating       TEXT,  
  duration     TEXT,  
  listed_in    TEXT,  
  description   TEXT  
);
```

Step 2 — Load the CSV file

```
COPY netflix_raw
```

```
FROM '/absolute/path/to/netflix_titles.csv'  
CSV HEADER QUOTE '"' ESCAPE '\\';
```

Step 3 — Create clean working table with parsed dates

```
CREATE TABLE netflix AS  
SELECT  
    show_id,  
    type,  
    title,  
    NULLIF(TRIM(director), '') AS director,  
    NULLIF(TRIM(cast_members), '') AS cast_members,  
    NULLIF(TRIM(country), '') AS country,  
    TO_DATE(TRIM(date_added),  
            'Month DD, YYYY') AS date_added,  
    release_year,  
    NULLIF(TRIM(rating), '') AS rating,  
    duration,  
    listed_in,  
    description  
FROM netflix_raw  
WHERE title IS NOT NULL;
```

Step 4 — Create the genre exploded VIEW

```
CREATE VIEW netflix_genres AS  
SELECT  
    show_id, type, country,  
    date_added, release_year, rating,  
    TRIM(genre) AS genre  
FROM netflix,  
    UNNEST(STRING_TO_ARRAY(listed_in, ',')) AS genre;
```

4. SQL TECHNIQUES APPLIED

This project deliberately exercises a curated set of intermediate-to-advanced SQL patterns. Each technique appears in multiple queries, giving repetition through context rather than dry drill.

Technique	How It Is Used in This Project
STRING_TO_ARRAY	Splits the comma-separated listed_in genre field into a proper array for UNNEST processing.
UNNEST	Explodes the genre array into individual rows — one row per genre per title — enabling genre-level aggregations.
CTE (WITH clause)	Organises multi-step logic in Q8 (YoY growth), Q14 (genre-per-year ranking), Q17 (dominant genre per country), Q24 (multi-type directors), and Q25 (dashboard).
Window Functions	LAG() for growth rate; SUM() OVER for running totals; RANK() OVER (PARTITION BY ...) for top-N ranking within groups.
CROSS JOIN / Self JOIN	Applied in Q15 to find genre co-occurrence — pairs every genre of a title against every other genre of the same title.
Date Parsing	TO_DATE() converts the raw text field (e.g. 'January 15, 2020') to a proper DATE; EXTRACT() and TO_CHAR() derive year/month components.
FILTER clause	Used in Q9, Q19 to pivot counts by type or country within a single GROUP BY query — a clean alternative to CASE WHEN.
SPLIT_PART	Extracts only the primary (first) country from the comma-separated country field for country-level aggregations.
STRING_AGG	Concatenates multiple distinct values (e.g. content types per director) into a single readable field in Q23.
NULLIF / TRIM	Converts empty strings to proper NULLs during the table creation step, ensuring accurate NULL-counts and cleaner joins.

5. QUERY INDEX (ALL 25 QUERIES)

The 25 queries are grouped into six thematic blocks, progressing from basic exploration to multi-window analytical dashboards.

#	Query Name	Technique Used	Theme / Block
Q1	Row count & type breakdown	Window Function	Block 1 — Exploration
Q2	NULL / missing value audit	FILTER + COUNT	Block 1 — Exploration
Q3	Earliest & latest content added	MIN / MAX	Block 1 — Exploration
Q4	Release year vs year added gap	Date Extraction	Block 1 — Exploration
Q5	Duplicate title check	GROUP BY + HAVING	Block 1 — Exploration
Q6	Titles added per year	Time Series	Block 2 — Trends
Q7	Monthly seasonality of additions	TO_CHAR + EXTRACT	Block 2 — Trends
Q8	Year-over-year growth rate	CTE + LAG()	Block 2 — Trends
Q9	Movies vs TV Shows added per year	FILTER pivot	Block 2 — Trends

Q10	Running cumulative library size	<i>SUM() OVER</i>	Block 2 — Trends
Q11	Explode genres (VIEW creation)	<i>UNNEST + STRING_TO_ARRAY</i>	Block 3 — Genre
Q12	Top 15 most common genres	<i>Aggregation</i>	Block 3 — Genre
Q13	Genre popularity by content type	<i>GROUP BY multi-dim</i>	Block 3 — Genre
Q14	Top 5 genres per year	<i>CTE + RANK()</i>	Block 3 — Genre
Q15	Genre co-occurrence matrix	<i>CROSS JOIN + Self JOIN</i>	Block 3 — Genre
Q16	Top content-producing countries	<i>SPLIT_PART</i>	Block 4 — Regional
Q17	Dominant genre per country	<i>CTE + RANK() PARTITION</i>	Block 4 — Regional
Q18	Countries with single content type	<i>HAVING + COUNT DISTINCT</i>	Block 4 — Regional
Q19	International vs US content ratio	<i>ILIKE + FILTER</i>	Block 4 — Regional
Q20	Rating distribution	<i>Window % calc</i>	Block 5 — Ratings
Q21	Average movie duration by genre	<i>UNNEST + CAST</i>	Block 5 — Ratings
Q22	TV Show season distribution	<i>Aggregation</i>	Block 5 — Ratings
Q23	Most prolific directors	<i>STRING_AGG</i>	Block 6 — Advanced
Q24	Directors with Movies + TV Shows	<i>CTE + COUNT DISTINCT</i>	Block 6 — Advanced
Q25	Full strategy dashboard per year	<i>CTE + Multi-window</i>	Block 6 — Advanced

6. QUERIES — BLOCK BY BLOCK

Block 1 — Data Exploration & Quality (Q1–Q5)

Establish baseline understanding: row counts, missing values, date ranges, and duplicates.

Q1 — Row count and type breakdown

```
SELECT type,
       COUNT(*) AS total,
       ROUND(COUNT(*) * 100.0 / SUM(COUNT(*) OVER ()), 2) AS pct
FROM   netflix
GROUP BY type;
```

Q2 — NULL / missing value audit

```
SELECT
  COUNT(*) FILTER (WHERE director IS NULL) AS missing_director,
  COUNT(*) FILTER (WHERE country IS NULL) AS missing_country,
  COUNT(*) FILTER (WHERE date_added IS NULL) AS missing_date_added,
  COUNT(*) FILTER (WHERE rating IS NULL) AS missing_rating,
  COUNT(*) FILTER (WHERE cast_members IS NULL) AS missing_cast
FROM netflix;
```

Q3 — Earliest and latest content added

```
SELECT MIN(date_added) AS first_added,
       MAX(date_added) AS last_added,
       MAX(date_added) - MIN(date_added) AS span_days
FROM   netflix;
```

Q4 — Licensed old content: release year vs year added gap

```
SELECT title, type, release_year,
       EXTRACT(YEAR FROM date_added)::INT AS year_added,
       EXTRACT(YEAR FROM date_added)::INT - release_year AS years_gap
FROM   netflix
WHERE  date_added IS NOT NULL
ORDER BY years_gap DESC
LIMIT 20;
```

Q5 — Duplicate title check

```
SELECT title, type, COUNT(*) AS cnt
FROM   netflix
GROUP BY title, type
HAVING COUNT(*) > 1
ORDER BY cnt DESC;
```

Block 2 — Content Addition Trends Over Time (Q6–Q10)

Time-series analysis of how Netflix has grown its catalogue year by year, including growth rates and cumulative totals.

Q6 — Titles added per year (Time Series)

```
SELECT EXTRACT(YEAR FROM date_added)::INT AS yr,
       COUNT(*) AS titles_added
FROM   netflix
WHERE  date_added IS NOT NULL
GROUP BY yr
ORDER BY yr;
```

Q7 — Monthly seasonality of content additions

```
SELECT TO_CHAR(date_added, 'Mon') AS month,
       EXTRACT(MONTH FROM date_added)::INT AS month_num,
       COUNT(*) AS titles_added
FROM   netflix
WHERE  date_added IS NOT NULL
GROUP BY month, month_num
ORDER BY month_num;
```

Q8 — Year-over-year growth rate (CTE + LAG)

```
WITH yearly AS (
    SELECT EXTRACT(YEAR FROM date_added)::INT AS yr,
           COUNT(*) AS titles
    FROM   netflix
    WHERE  date_added IS NOT NULL
    GROUP BY yr
),
growth AS (
    SELECT yr, titles,
           LAG(titles) OVER (ORDER BY yr) AS prev_year,
           ROUND(
               (titles - LAG(titles) OVER (ORDER BY yr)) * 100.0 /
               NULLIF(LAG(titles) OVER (ORDER BY yr), 0), 2) AS yoy_pct
    FROM yearly
)
SELECT * FROM growth ORDER BY yr;
```

Q9 — Movies vs TV Shows added per year (pivot)

```
SELECT EXTRACT(YEAR FROM date_added)::INT AS yr,
       COUNT(*) FILTER (WHERE type = 'Movie') AS movies,
       COUNT(*) FILTER (WHERE type = 'TV Show') AS tv_shows
FROM   netflix
WHERE  date_added IS NOT NULL
GROUP BY yr
ORDER BY yr;
```


Q10 — Running cumulative library size

```

SELECT  EXTRACT(YEAR FROM date_added)::INT          AS yr,
        COUNT(*)                                   AS added_this_year,
        SUM(COUNT(*)) OVER (ORDER BY
            EXTRACT(YEAR FROM date_added))          AS cumulative_total
FROM    netflix
WHERE   date_added IS NOT NULL
GROUP  BY yr
ORDER  BY yr;

```

Block 3 — Genre Analysis with UNNEST (Q11–Q15)

Uses STRING_TO_ARRAY and UNNEST to decompose the comma-separated genre field and perform genre-level aggregations and co-occurrence analysis.

Q11 — Top 15 most common genres overall

```

SELECT genre, COUNT(*) AS total
FROM    netflix_genres
GROUP  BY genre
ORDER  BY total DESC
LIMIT  15;

```

Q12 — Genre popularity by content type

```

SELECT genre, type, COUNT(*) AS cnt
FROM    netflix_genres
GROUP  BY genre, type
ORDER  BY genre, cnt DESC;

```

Q13 — Top 5 genres per year (CTE + RANK)

```

WITH genre_year AS (
    SELECT EXTRACT(YEAR FROM date_added)::INT AS yr,
           genre,
           COUNT(*)                           AS cnt
    FROM    netflix_genres
    WHERE   date_added IS NOT NULL
    GROUP  BY yr, genre
),
ranked AS (
    SELECT *,
           RANK() OVER (PARTITION BY yr ORDER BY cnt DESC) AS rnk
    FROM    genre_year
)
SELECT yr, genre, cnt
FROM    ranked
WHERE   rnk <= 5
ORDER  BY yr, rnk;

```

Q14 — Genre co-occurrence matrix (CROSS JOIN / Self JOIN)

```

WITH exploded AS (
    SELECT show_id,
           TRIM(g) AS genre
    FROM   netflix,
           UNNEST(STRING_TO_ARRAY(listed_in, ',')) g
)
SELECT  a.genre    AS genre_a,
        b.genre    AS genre_b,
        COUNT(*)   AS co_occurrences
FROM    exploded a
JOIN    exploded b
        ON  a.show_id = b.show_id
        AND a.genre   < b.genre
GROUP   BY a.genre, b.genre
ORDER   BY co_occurrences DESC
LIMIT   20;

```

Block 4 — Country & Regional Analysis (Q15–Q19)

Isolates primary production country and compares genre preferences, content type exclusivity, and international vs US library share.

Q15 — Top content-producing countries

```

SELECT TRIM(SPLIT_PART(country, ',', 1)) AS primary_country,
       COUNT(*)                          AS titles
FROM   netflix
WHERE  country IS NOT NULL
GROUP  BY primary_country
ORDER  BY titles DESC
LIMIT  15;

```

Q16 — Dominant genre per country (CTE + RANK PARTITION)

```

WITH country_genre AS (
    SELECT TRIM(SPLIT_PART(n.country, ',', 1)) AS country,
           TRIM(g.genre)                       AS genre,
           COUNT(*)                           AS cnt
    FROM   netflix n,
           UNNEST(STRING_TO_ARRAY(n.listed_in, ',')) AS g(genre)
    WHERE  n.country IS NOT NULL
    GROUP  BY country, genre
),
ranked AS (
    SELECT *,
           RANK() OVER (PARTITION BY country ORDER BY cnt DESC) AS rnk
    FROM   country_genre
)
SELECT country, genre, cnt

```

```
FROM ranked
WHERE rnk = 1 AND cnt > 5
ORDER BY cnt DESC;
```

Q17 — Countries that produce only Movies or only TV Shows

```
SELECT TRIM(SPLIT_PART(country, ',', 1)) AS country,
       COUNT(DISTINCT type)              AS type_count,
       STRING_AGG(DISTINCT type, ', ')   AS types
FROM   netflix
WHERE  country IS NOT NULL
GROUP BY country
HAVING COUNT(DISTINCT type) = 1
ORDER BY country;
```

Q18 — International vs US content ratio by year

```
SELECT EXTRACT(YEAR FROM date_added)::INT AS yr,
       COUNT(*) FILTER (WHERE country ILIKE '%United States%') AS
us_titles,
       COUNT(*) FILTER (WHERE country NOT ILIKE '%United States%'
                          AND country IS NOT NULL) AS
intl_titles
FROM   netflix
WHERE  date_added IS NOT NULL
GROUP BY yr
ORDER BY yr;
```

Block 5 — Ratings & Duration Analysis (Q19–Q22)

Examines content rating distributions, average movie runtimes by genre, and TV season counts.

Q19 — Rating distribution with percentage share

```
SELECT rating,
       COUNT(*) AS total,
       ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS pct
FROM   netflix
WHERE  rating IS NOT NULL
GROUP BY rating
ORDER BY total DESC;
```

Q20 — Average movie duration by genre

```
SELECT TRIM(g) AS genre,
       ROUND(AVG(REPLACE(duration, ' min', '')::INT), 1) AS avg_minutes,
       MIN(REPLACE(duration, ' min', '')::INT) AS min_min,
       MAX(REPLACE(duration, ' min', '')::INT) AS max_min
FROM   netflix,
       UNNEST(STRING_TO_ARRAY(listed_in, ',')) AS g
WHERE  type = 'Movie' AND duration ILIKE '%min%'
```

```
GROUP BY genre
ORDER BY avg_minutes DESC;
```

Q21 — TV Show season distribution

```
SELECT duration AS seasons, COUNT(*) AS shows
FROM netflix
WHERE type = 'TV Show'
GROUP BY duration
ORDER BY shows DESC;
```

Block 6 — Directors, Cast & Dashboard (Q22–Q25)

Advanced aggregations on directors, multi-type contributors, and a comprehensive year-level content strategy dashboard query.

Q22 — Most prolific directors on Netflix

```
SELECT director,
       COUNT(*) AS titles,
       STRING_AGG(DISTINCT type, ', ') AS content_types
FROM netflix
WHERE director IS NOT NULL
GROUP BY director
ORDER BY titles DESC
LIMIT 15;
```

Q23 — Directors who directed both Movies AND TV Shows

```
WITH director_types AS (
  SELECT director,
         COUNT(DISTINCT type) AS type_count,
         COUNT(*) AS total_titles
  FROM netflix
  WHERE director IS NOT NULL
  GROUP BY director
)
SELECT director, total_titles
FROM director_types
WHERE type_count = 2
ORDER BY total_titles DESC
LIMIT 10;
```

Q24 — Cast member frequency analysis

```
SELECT TRIM(actor) AS actor,
       COUNT(*) AS appearances
FROM netflix,
     UNNEST(STRING_TO_ARRAY(cast_members, ',')) AS actor
WHERE cast_members IS NOT NULL
GROUP BY actor
```

```
ORDER BY appearances DESC
LIMIT 20;
```

Q25 — Full content strategy dashboard (CTE + multi-window)

```
WITH yearly_stats AS (
    SELECT
        EXTRACT(YEAR FROM date_added)::INT AS yr,
        COUNT(*) AS total_titles,
        COUNT(*) FILTER (WHERE type = 'Movie') AS movies,
        COUNT(*) FILTER (WHERE type = 'TV Show') AS tv_shows,
        COUNT(DISTINCT TRIM(SPLIT_PART(country, ',', 1))) AS
unique_countries,
        ROUND(AVG(
            CASE WHEN type = 'Movie'
                THEN REPLACE(duration, ' min', '')::INT END), 1) AS avg_movie_mins
    FROM    netflix
    WHERE   date_added IS NOT NULL
    GROUP   BY yr
)
SELECT *,
    SUM(total_titles) OVER (ORDER BY yr) AS cumulative_library,
    ROUND(movies * 100.0 /
        NULLIF(total_titles, 0), 1) AS movie_share_pct
FROM    yearly_stats
ORDER   BY yr;
```

7. EXPECTED INSIGHTS & OUTCOMES

Running all 25 queries against the Kaggle Netflix dataset should reveal the following patterns:

Content Addition Trends

- Netflix's biggest library growth occurred between 2016 and 2019, driven by an aggressive originals strategy.
- Content additions typically spike in Q4 (October–December), suggesting an end-of-year subscriber acquisition push.
- Movies consistently outnumber TV Shows by approximately 2:1 across the catalogue.

Genre Insights

- 'International Movies' and 'Dramas' are the two most prolific genre tags globally.
- The United States is dominated by Drama and Documentary content; India heavily favours International Movies with Action & Adventure.
- 'Dramas' and 'International Movies' have the highest genre co-occurrence rate — they nearly always appear together.

Ratings & Duration

- TV-MA is the dominant content rating, signalling Netflix's strategic focus on adult audiences.
- Stand-up Comedy and Children's films are outliers on duration — lowest and highest average runtimes respectively.
- Most TV shows run for 1–2 seasons, indicating Netflix's lean toward limited series formats.

Country & Directors

- The United States, India, and the United Kingdom are the top three producing countries by volume.
- Rajiv Chilaka (Indian animation director) leads the prolific directors table.
- A significant portion of licensed content was produced 5–15 years before its Netflix addition date.

8. SKILLS & LEARNING OUTCOMES

Completing this project builds the following demonstrable SQL competencies:

1. Fluency with `STRING_TO_ARRAY` and `UNNEST` for normalising denormalised, comma-packed columns.
2. Constructing multi-step CTEs that chain aggregations and window functions cleanly.
3. Using `LAG()`, `RANK()`, and `SUM() OVER` for time-series and top-N analytical queries.
4. Parsing and manipulating raw date strings into proper `DATE` types using `TO_DATE()` and `EXTRACT()`.
5. Self-joining exploded tables to compute co-occurrence or pair-wise relationships.
6. Pivoting aggregated counts with the `FILTER` clause as a clean alternative to `CASE WHEN`.
7. Designing a reusable `VIEW` (`netflix_genres`) that simplifies downstream queries.

- Communicating analytical findings through structured SQL query documentation.

9. EXTENSION IDEAS

For further practice, the following extensions can be added to this project:

- Join with an OWID (Our World in Data) CO₂ dataset on country to explore if high-content countries also rank high on carbon footprint — linking media consumption to environmental data.
- Connect to a TMDB or IMDB ratings CSV and analyse whether Netflix's licensed catalogue is predominantly highly-rated or mid-tier content.
- Build a stored procedure that accepts a genre string and returns the top 10 titles of that genre added in the last three years.
- Use RECURSIVE CTEs to model a simple content recommendation path: titles sharing two or more genres.
- Visualise Q25 (dashboard query) output in Python (matplotlib/seaborn) or Tableau for a presentation-ready chart.

10. REFERENCES & RESOURCES

Primary Dataset	kaggle.com/datasets/shivamb/netflix-shows
PostgreSQL Docs	postgresql.org/docs/current/
STRING_TO_ARRAY	postgresql.org/docs/current/functions-array.html
Window Functions	postgresql.org/docs/current/tutorial-window.html
UNNEST Reference	postgresql.org/docs/current/functions-array.html#FUNCTIONS-ARRAY-TABLE